IKT215 Assignment 1: Gaussian Naïve Bayes Classifier

Student: Gomery K. Wanjiru
Course: IKT215

Disclaimer on Generative AI

In accordance with UiA's guidelines, I declare that I used Generative AI (Google Gemini) to assist in generating the interactive Plotly visualization code, and formatting the mathematical equations within this report. The part of the Gaussian Naïve Bayes implementation and the specific analysis of results etc are my own work.

Introduction

This assignment focuses on implementing a Gaussian Naïve Bayes (GNB) classifier from scratch. The main goal was the probabilistic foundations of the algorithm, specifically using Bayes' Theorem and the assumption of independence among features. I implemented the algorithm in Python, tested it on a synthetic binary dataset, and extended it for multi-class classification using the Iris dataset.

Methodology

I built the classifier as a Python class `GaussianNBClassifier`.

Mathematical Foundation

The classifier assigns a class y that maximizes the posterior probability $P(Y = y|X = x)$. Following the "naïve" independence assumption, this is proportional to:

$$P(Y = y) \prod_{i=1}^{d} P(x_i|Y = y) \tag{1}$$

where $P(Y = y)$ is the prior probability (calculated as the frequency of the class in the training data) and $P(x_i|Y = y)$ is the likelihood.

Implementation

- **Training (`fit`):** The model calculates the mean (μ) and variance (σ^2) for every feature for each specific class. This defines the Gaussian distribution for each attribute. (assuming the datasett is gaussian distributed)
- **Prediction (`predict`):** I used the Gaussian Probability Density Function (PDF) to calculate feature likelihoods (again this formula is only valid if the data is gaussian distributed. different distributions would require different likelihood functions):

$$P(x_i|Y = y_j) = \frac{1}{\sigma_{i,j}\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x_i - \mu_{i,j}}{\sigma_{i,j}}\right)^2\right) \tag{2}$$

  The total likelihood is the product of these individual feature probabilities. The model then selects the class with the highest product of the prior and total likelihood.

Key Findings and Results

Task 1: Synthetic Binary Dataset

Using 1000 samples and 2 features, the model performed effectively with an accuracy of approximately 90-93%.

Task 3: Iris Dataset (Multi-class)

- **Scenario A (2 Sepal Features):** Accuracy was roughly 90%. The overlap in sepal dimensions makes clear separation difficult (see figure 1 and 2).
- **Scenario B (All 4 Features):** Performance jumped to nearly 100%. It seems petal dimensions provide higher information for the model to be able to discriminate more between the classes. Since it often predicts with 100% accuracy (see figure 3 and 4), it could mean the model is overfit and i expect it to perform worse on unseen data.

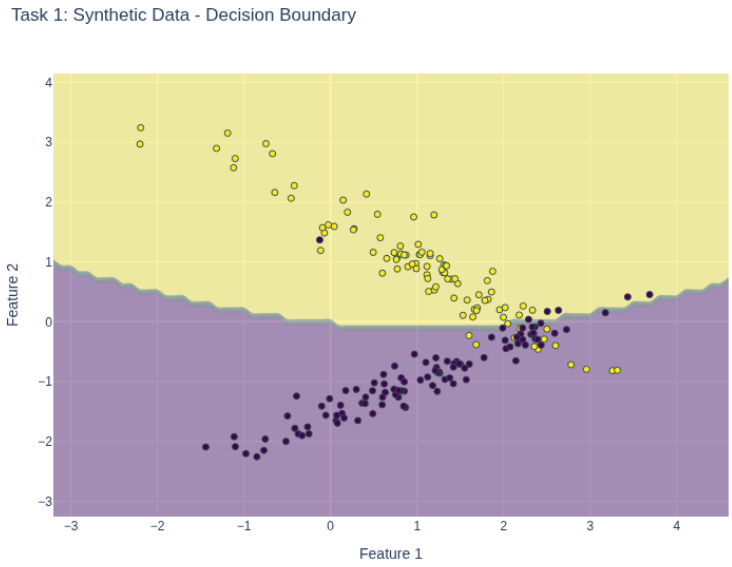

Figure 1: Decision Boundary for Synthetic Binary Classification

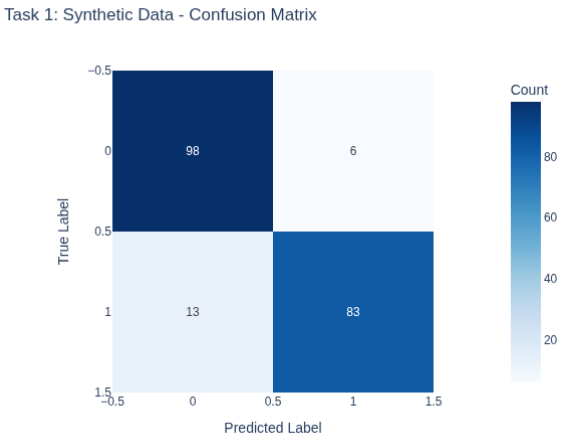

Figure 2: Confusion Matrix for Synthetic Data

Challenges and Solutions

Numerical Stability: I followed the instructions and used the direct product, but it's risky because multiplying all those tiny decimals can lead to underflow. Usually, you'd just sum the logs to keep the math stable, especially with bigger datasets.

Independence Assumption: Features in the Iris dataset are naturally correlated. However, the GNB model still predicted relatively accurately. This proves the algorithm's effectiveness even when the independence assumption is violated. This is perhaps the biggest assumption that make this model naive, so seeing it perform this well is impressive.

Conclusion

I implemented the GNB classifier from scratch. The results demonstrate that while GNB relies on a simple independence assumption, it is a computationally efficient and highly effective algorithm

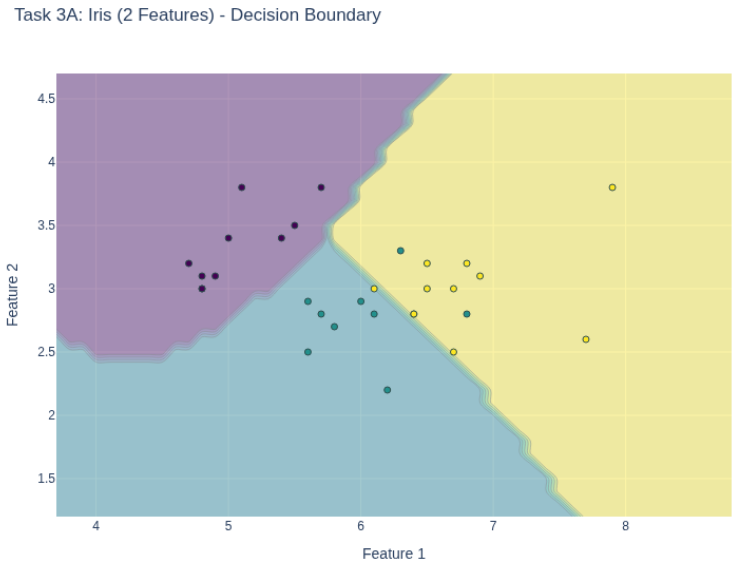

Figure 3: Iris Decision Boundary (Sepal Features)

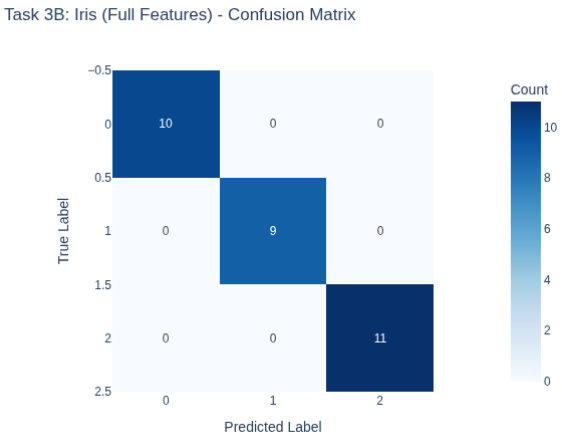

Figure 4: Confusion Matrix for Full Iris Dataset (4 Features)

for both binary and multi-class classification tasks.